

Contents

CODE_WALKTHROUGH.md	1
Подробный разбор кода проекта	1
1. model.py — расчётное ядро	1
2. utils.py — вспомогательные функции	3
3. app.py — интерфейс Streamlit	4
4. Сводная таблица: файл → блок → что делает → как объяснить	6

CODE_WALKTHROUGH.md

Подробный разбор кода проекта

Проект: «Монте-Карло оценка девелоперского проекта: NPV и корреляция рисков»

Автор: Левшин Даниил Дмитриевич

Разбор опирается на реальные имена функций и переменных из кода. Цель — чтобы можно было открыть любой файл и уверенно объяснить, что делает каждый блок.

Порядок чтения проекта: **model.py** → **utils.py** → **app.py**. Сначала математика, затем сервис, затем интерфейс, который их связывает.

1. model.py — расчётное ядро

1.1. Назначение файла

Вся «математика» проекта: расчёт NPV, симуляция Монте-Карло, генерация коррелированных рисков, риск-метрики и чувствительность. Файл **не зависит от Streamlit** и может использоваться отдельно (в скриптах, тестах, ноутбуках). Внизу есть самотест в блоке `if __name__ == "__main__":`.

1.2. Ключевые импорты

- `numpy as np` — векторные вычисления, линейная алгебра (Холецкий), генерация случайных чисел;
- `math` — `erf` для функции нормального распределения без SciPy;
- `dataclasses.dataclass` — структуры параметров;
- `try: from scipy.stats import norm` — SciPy используется, если установлен; иначе включается NumPy-fallback (`_norm_cdf` через `math.erf`, `_norm_ppf` — аппроксимация Акклама). Флаг `HAS SCIPY`.

1.3. Основные переменные и структуры

- `RISK_LABELS` — список из 5 названий факторов риска (порядок важен — он же в матрице).
- `DEFAULT_CORRELATION` — базовая корреляционная матрица 5×5 (numpy array).

- @dataclass ProjectParams — базовые параметры проекта (инвестиции, выручка, затраты, OPEX, ставка, горизонт, профили потоков, расходы на содержание при задержке).
- @dataclass RiskParams — параметры предельных распределений факторов.

1.4. Основные функции (в порядке логики)

- `_norm_cdf`, `_norm_ppf` — функция и обратная функция стандартного нормального распределения.
- `_triangular_ppf(u, a, c, b)` — обратная функция треугольного распределения (замкнутая формула, с защитой от вырожденного случая).
- `_discrete_ppf(u, values, probs)` — обратная функция дискретного распределения (нормирует вероятности, назначает исходы по кумулятивным интервалам).
- `calculate_base_npv(params)` — детерминированная NPV (вызывает `calculate_npv` при нулевых факторах).
- `calculate_npv(...)` — векторизованный расчёт NPV для массива сценариев.
- `_profile(profile, T, front)` — приводит профиль потоков по годам к нужному горизонту.
- `generate_correlated_risks(...)` — гауссова копула + Холецкий.
- `run_monte_carlo(...)` — полная симуляция одной модели.
- `run_both_models(...)` — обе модели (коррелированная и независимая).
- `calculate_risk_metrics(...)` — VaR, ES, P(NPV<0), перцентили и т.д.
- `calculate_sensitivity(...)` — корреляция факторов с NPV.
- `scenario_npv(...)` — NPV одного заданного сценария (скаляры).

1.5. Что в каком порядке происходит при расчёте

`run_both_models` → дважды `run_monte_carlo` (с корреляцией и без) → `generate_correlated_risks` (генерация факторов) → `calculate_npv` (NPV каждого сценария) → `calculate_risk_metrics` + `calculate_sensitivity` (агрегаты).

1.6. Ключевые блоки кода

Гауссова копула и Холецкий (`generate_correlated_risks`):

```
rng = np.random.default_rng(seed)
z = rng.standard_normal((n, 5))
if use_correlation:
    L = np.linalg.cholesky(corr_matrix)    #  $\Sigma = L \cdot L^T$ 
    z = z @ L.T                            # коррелированные нормальные
u = _norm_cdf(z)                           # → равномерные
u = np.clip(u, 1e-10, 1 - 1e-10)          # стабилизация краёв
cost = _triangular_ppf(u[:, 0], *risk.cost_growth)
price = _triangular_ppf(u[:, 1], *risk.price_change)
delay = _discrete_ppf(u[:, 2], risk.delay_values, risk.delay_probs)
reg = _triangular_ppf(u[:, 3], *risk.reg_cost)
absorption = _triangular_ppf(u[:, 4], *risk.absorption)
```

Расчёт NPV (`calculate_npv`):

```
delay_yr = delay_months / 12.0
rev_total = params.base_revenue * (1.0 + price_change)
```

```

cons_total = params.base_construction * (1.0 + cost_growth)
# ... перераспределение выручки по absorption ...
npv = np.full(len(cost_growth), -params.initial_investment)
for t in range(1, T + 1):
    rev_part = rev_total * rev_frac_mat[:, t-1]
    cost_part = cons_total * cons_frac[t-1] + params.opex * opex_frac[t-1]
    npv += rev_part / (1.0 + r) ** (t + delay_yr) # выручка с учётом задержки
    npv += (-cost_part) / (1.0 + r) ** t
extra = reg_cost + params.delay_carry_per_year * delay_yr
npv += (-extra) / (1.0 + r) ** 1

```

VaR и Expected Shortfall (calculate_risk_metrics):

```

p5 = np.percentile(npv, 5) # VaR (5%)
k = max(1, int(0.05 * n))
es = np.sort(npv)[:k].mean() # среднее худших 5%
prob_neg = np.mean(npv < 0) * 100 # вероятность убытка, %

```

Чувствительность (calculate_sensitivity):

```

corr = np.corrcoef(factors[:, j], npv)[0, 1] # Пирсон, фактор vs NPV
# затем сортировка по abs(corr)

```

1.7. Как объяснить файл преподавателю

«model.py — это вся математика отдельно от интерфейса. Здесь параметры проекта и рисков описаны структурами ProjectParams/RiskParams. Сердце — generate_correlated_risks, где через разложение Холецкого корреляционной матрицы я превращаю независимые нормальные величины в коррелированные, а потом через обратные функции распределений получаю значения факторов. calculate_npv векторно считает NPV всех сценариев сразу, а calculate_risk_metrics даёт VaR, ES и вероятность убытка».

2. utils.py — вспомогательные функции

2.1. Назначение файла

Сервисный слой между математикой и интерфейсом: форматирование чисел, построение таблиц, проверка корректности корреляционной матрицы, генерация текстового инвестиционного вывода. Тоже не зависит от Streamlit.

2.2. Ключевые импорты

- numpy as np, pandas as pd — таблицы и линейная алгебра;
- from model import RISK_LABELS — единые подписи факторов.

2.3. Основные функции

Форматирование: - fmt_money(value, digits=1) — «1 234,5 млн руб.» (пробел-разделитель, запятая); - fmt_num(value, digits=1) — число без единиц; - fmt_pct(value, digits=1) — процент (вход уже в процентах).

Проверка матрицы: - `_nearest_psd(matrix)` — ближайшая положительно полуопределённая матрица: симметризация → `eigh` → клиппинг отрицательных собственных значений → нормировка диагонали к 1; - `validate_correlation_matrix(matrix, fallback)` — проверяет форму, диапазон `[-1,1]`, симметрию, диагональ, положительную определённость через `np.linalg.cholesky`; при некорректности возвращает скорректированную (или базовую) матрицу. Возвращает кортеж `(matrix, is_valid, message)`; - `is_positive_definite(matrix)` — быстрая проверка через Холецкого.

Таблицы: - `metrics_table(metrics_corr, metrics_ind)` — сравнительная таблица показателей двух моделей; - `sensitivity_table(sensitivity)` — таблица чувствительности; - `correlation_dataframe(matrix)` — матрица с подписями факторов; - `scenario_table(scenarios)` — таблица сценариев.

Текстовый вывод: - `generate_investment_verdict(metrics_corr, metrics_ind)` — классифицирует проект (`attractive/moderate/risky`) по средней NPV и `prob_neg`, формирует `headline`, `body` и `effect` (эффект учёта корреляции — сравнение двух моделей).

2.4. Что в каком порядке происходит

`app.py` сначала вызывает `validate_correlation_matrix` (до симуляции), затем после расчёта — `metrics_table`, `sensitivity_table` и `generate_investment_verdict` для отображения.

2.5. Как объяснить файл преподавателю

«`utils.py` — это сервис интерфейса. Самая содержательная функция — `validate_correlation_matrix`: она проверяет, что введённая пользователем матрица математически корректна (положительно определена), и если нет — строит ближайшую корректную, чтобы симуляция не сломалась. Остальное — форматирование под русский стандарт, сборка таблиц и автоматический текстовый вывод по результатам».

3. `app.py` — интерфейс Streamlit

3.1. Назначение файла

Только пользовательский интерфейс и визуализация. Вся логика расчётов делегируется в `model.py`, форматирование и проверки — в `utils.py`.

3.2. Ключевые импорты

- `streamlit as st` — интерфейс;
- `numpy as np, pandas as pd` — данные;
- `plotly.graph_objects as go` — интерактивные графики;
- `import model, import utils` — расчётное ядро и сервис.

3.3. Основные переменные и хелперы

- `st.set_page_config(...)` — конфигурация страницы (вызывается первым);

- палитра цветов (BLUE, RED, GREEN ...), PLOTLY_FONT, CSS-блок;
- `metric_card(label, value, color)` — HTML KPI-карточка;
- функции графиков: `fig_histogram`, `fig_cdf`, `fig_tornado`, `fig_heatmap`, `fig_compare_hist`;
- `project = model.ProjectParams(...)` — собирается из полей сайдбара;
- `risk = model.RiskParams(...)` — собирается из полей раздела 3;
- `st.session_state["results"]` — хранилище результатов симуляции.

3.4. Порядок: 10 разделов интерфейса

1. **Описание модели** — текст + блоки «Целевая аудитория» и «Практический СМЫСЛ».
2. **Ввод параметров** — таблица параметров и карточка базовой NPV (`calculate_base_npv`).
3. **Настройка факторов риска** — поля ввода → `RiskParams`.
4. **Корреляционная матрица** — `st.data_editor` → `validate_correlation_matrix` → `heatmap`.
5. **Запуск симуляции** — кнопка `run` → `run_both_models` → `session_state`.
6. **Результаты Monte Carlo** — KPI-карточки + гистограмма.
7. **Сравнение моделей** — наложенные гистограммы + CDF + `metrics_table`.
8. **Чувствительность** — `tornado` + `sensitivity_table`.
9. **Инвестиционный вывод** — `generate_investment_verdict`.
10. **Методология** — формулы (`st.latex`) и описание.

3.5. Ключевые блоки кода

Запуск симуляции и `session_state`:

```
if run:
    with st.spinner("Выполняется ... итераций Монте-Карло..."):
        try:
            results = model.run_both_models(project, risk, corr_matrix,
                                             n=int(n_iter), seed=int(seed))
            st.session_state["results"] = results
        except Exception as exc:
            st.error(f"Ошибка при расчёте: {exc}")
if "results" not in st.session_state:
    st.info("Задайте параметры ... и нажмите «Запустить симуляцию».")
    st.stop()
```

Проверка матрицы:

```
corr_input = edited.to_numpy(dtype=float)
corr_matrix, is_valid, corr_msg = utils.validate_correlation_matrix(
    corr_input, fallback=model.DEFAULT_CORRELATION)
if is_valid: st.success("✓ " + corr_msg)
else:       st.warning("⚠ " + corr_msg)
```

3.6. Как объяснить файл преподавателю

«app.py — это лицо проекта. Вот **здесь** (сайдбар) пользователь задаёт параметры проекта и симуляции; **здесь** (разделы 3–4) — риски и корреляционную матрицу, которая сразу проверяется на корректность; по кнопке вызывается **расчётная**

модель `model.run_both_models`, результат кладётся в `session_state`, чтобы не пересчитывать его при каждом действии; **здесь** (разделы 6–9) результаты рисуются графиками Plotly, а в конце формируется автоматический инвестиционный вывод».

4. Сводная таблица: файл → блок → что делает → как объяснить

Файл	Функция / блок	Что делает	Как объяснить преподавателю
model.py	ProjectParams, RiskParams	Группируют параметры проекта и рисков	«Структуры данных — вместо десятков переменных два понятных объекта с значениями по умолчанию»
model.py	calculate_base_npv	Точечная (детерминированная) NPV	«Классический NPV — отправная точка»
model.py	calculate_npv	Векторный NPV всех сценариев	«Формула DCF, применённая сразу ко всем 10 000 сценариям»
model.py	generate_correlated_normals	Копия + Холецкий	«Здесь риски получают заданную корреляцию»
model.py	np.linalg.cholesky / z @ L.T	Впечатывание корреляции	«Разложение Холецкого превращает независимые нормальные в коррелированные»
model.py	_triangular_ppf, _discrete_ppf	Обратные функции распределений	«Переводят равномерные числа в значения факторов»
model.py	run_both_models	Обе модели сразу	«Считаю независимую и коррелированную модели для сравнения»
model.py	calculate_risk_metrics	MaR, ES, P(NPV<0)	«Превращаю распределение в риск-метрики»
model.py	calculate_sensitivity	Корреляция факторов с NPV	«Отвечаю, какой фактор сильнее двигает NPV»

Файл	Функция / блок	Что делает	Как объяснить преподавателю
utils.py	validate_correlation	Проверка/коррекция матрицы (PSD)	«Гарантирую математическую корректность матрицы»
utils.py	fmt_money/fmt_pct/fmt_pct	Универсальное форматирование	«Единый аккуратный формат чисел»
utils.py	metrics_table, sensitivity_table	Таблицы для UI	«Готовлю данные к отображению»
utils.py	generate_investment_text	Текстовый вывод	«Автоматически формулирую инвестиционный вердикт»
app.py	st.set_page_config, CSS	Оформление страницы	«Настройка внешнего вида»
app.py	st.sidebar блок	Ввод базовых параметров	«Здесь пользователь меняет параметры проекта»
app.py	st.data_editor	Редактирование матрицы	«Пользователь правит корреляции прямо в таблице»
app.py	run + session_state	Запуск и хранение результата	«Считаю по кнопке и кэширую результат между перерисовками»
app.py	fig_histogram/fig_cdf/fig_factor_plot/fig_heatmap/fig_compare_hist	Графики	«Визуализация распределения, сравнения и чувствительности»